

Mahendra Institute of Engineering and Technology

Department of Computer Science and Engineering

An Intelligent Hybrid GCN-LSTM model for Energy Stock Price Forecasting with Temporal Dynamics and Inter-stock correlation

Guided by,

Mrs. BANUPPRIYA P, B.E., ME.,
Assistant Professor / CSE

Team Member's

611522104106- YOKESH KUMAR E
611522104011- ARUN KUMAR R
611522104069- RANJITH KUMAR S
611522104063- POOVARASAN M

OUTLINE

- Abstract .
- Objective .
- Literature Survey .
- Existing System .
- Drawbacks of Existing System .
- Proposed System .
- Advantages of Proposed System .
- System Architecture .
- Hardware and Software Requirements .
- List of Modules .
- List of Modules (DFD) .
- Result/Output .
- Output Screenshots .
- Conclusion and Future Enhancement .
- References .

ABSTRACT

- Stock price prediction is a challenging task due to the non-linear and dynamic nature of financial markets.

- Traditional prediction techniques often fail to capture complex market behaviour and long-term dependencies present in historical data.

- Deep learning models such as Long Short-Term Memory (LSTM) networks are effective in learning temporal patterns from time-series stock data, but they treat each stock independently and ignore relationships between companies.

- Additionally Technical Indicators like Relative Strength Index indicates the overbought and oversold rates of the stocks.

OBJECTIVE

- To study and analyse historical stock price data of energy sector companies in order to understand their time-dependent behaviour and price movement patterns over different periods.
- To identify and model inter-stock relationships among energy sector companies using graph-based representations, where each company is treated as a node connected based on correlation.
- To design and implement a hybrid deep learning framework that integrates Graph Convolutional Networks (GCN) with Long Short-Term Memory (LSTM) networks and RSI Technical Indicators for effective stock price forecasting.
- To evaluate the performance of the proposed hybrid model by comparing it with traditional and standalone deep learning models such as LSTM.

LITERATURE SURVEY

TITLE	AUTHOR AND YEAR	TECHNOLOGY USED	MERITS	DEMERITS
Stock Price Prediction Using GCN and RNN	Antony Taurshia etal., (2025)	GCN + RNN(LSTM)	Integrates spatial (inter-stock) and temporal information.	Assumes static inter-stock relationships.
Comparative DL Models for Stock Prediction	Moumita Barua et al.(2024)	CNN, LSTM, GRU	Compares multiple deep learning models for stock prediction.	Ignores inter-stock and sector-level relationships.
Stock Price Prediction using LSTM	Sagar Shinde; Lalit Kumar Wadhwa (2023)	LSTM	Captures long-term temporal patterns in stock data.	Ignores inter-stock relationships and external factors.
Stock Price Prediction Using GCN	Li, X., Wang, J., Zhang, Y. (2022)	GCN	Models inter-stock relationships and temporal dependencies jointly	Graph structure is static and computationally expensive
Stock Price Prediction Using Machine Learning	Daniel Jonathan (2021)	ML and ARIMA	Handles the linear-time series Stock price	Lags with Dynamic Time Warping (Non-linear Patterns)

EXISTING SYSTEM

- Traditional stock price prediction systems rely on statistical and machine learning models basic regression techniques. These models assume linear relationships and are not effective in capturing complex market behaviour(DTW)

- Deep learning models like Recurrent Neural Networks (RNN) and LSTM are used to analyse historical stock price data and learn temporal patterns. These models improve prediction accuracy compared to traditional methods.

- Some recent approaches use Graph Convolutional Networks (GCN) to model relationships between multiple stocks by representing companies as nodes connected through correlations.

- A hybrid GCN–LSTM model is used, where GCN captures inter-stock relationships and RNN models temporal price movements.

DRAWBACKS OF EXISTING SYSTEM

- Most existing systems use basic RNN architectures, which suffer from vanishing gradient problems and are not effective in learning long-term dependencies in stock price time-series data.
- The inter-stock relationship graph used in GCN-based models is usually static, meaning it does not adapt to changing market conditions over time.
- Many models treat each stock independently or give limited importance to sector-level dependencies, reducing prediction reliability for related companies.
- Existing systems often lack robustness during market volatility and do not generalize well across different time periods.
- No Technical Indicators included along the GCN-LSTM models

PROPOSED SYSTEM

- In the proposed system, energy sector companies are represented as nodes in a graph, and edges are formed based on correlation between their historical stock prices.

- A Graph Convolutional Network (GCN) is used to learn inter-stock relationships by aggregating information from correlated energy companies.

- This project uses Long Short-Term Memory (LSTM) networks as an extension of RNN model temporal dependencies more effectively.

- RSI as Technical Indicator improves the Prediction accuracy rate by indicating the **OVERBOUGHT** and **OVERSOLD** rate.

- The hybrid GCN–LSTM framework jointly learns spatial (inter-stock correlation) and temporal (time-series behaviour) features to improve stock price prediction.

ADVANTAGES OF PROPOSED SYSTEM

- **Sees the full picture:** Most models look at one stock at a time. Our system looks at all 28 energy stocks together and understands how they affect each other — giving it a much bigger view of the market.

- **Better memory for past trends:** The LSTM part of our model is very good at remembering what happened weeks or even months ago and using that to make a smarter prediction today — older models tend to forget this quickly.

- **RSI gives extra insight:** By adding the RSI indicator, our model can tell when a stock has been rising too fast or falling too much, and uses that signal to predict the next move more accurately.

- **More reliable under market ups and downs:** Because our system considers both how stocks relate to each other and what their own history looks like, it handles sudden market changes better than simpler models that only look at one thing at a time.

HARDWARE AND SOFTWARE REQUIREMENTS

- **HARDWARE REQUIREMENTS :**

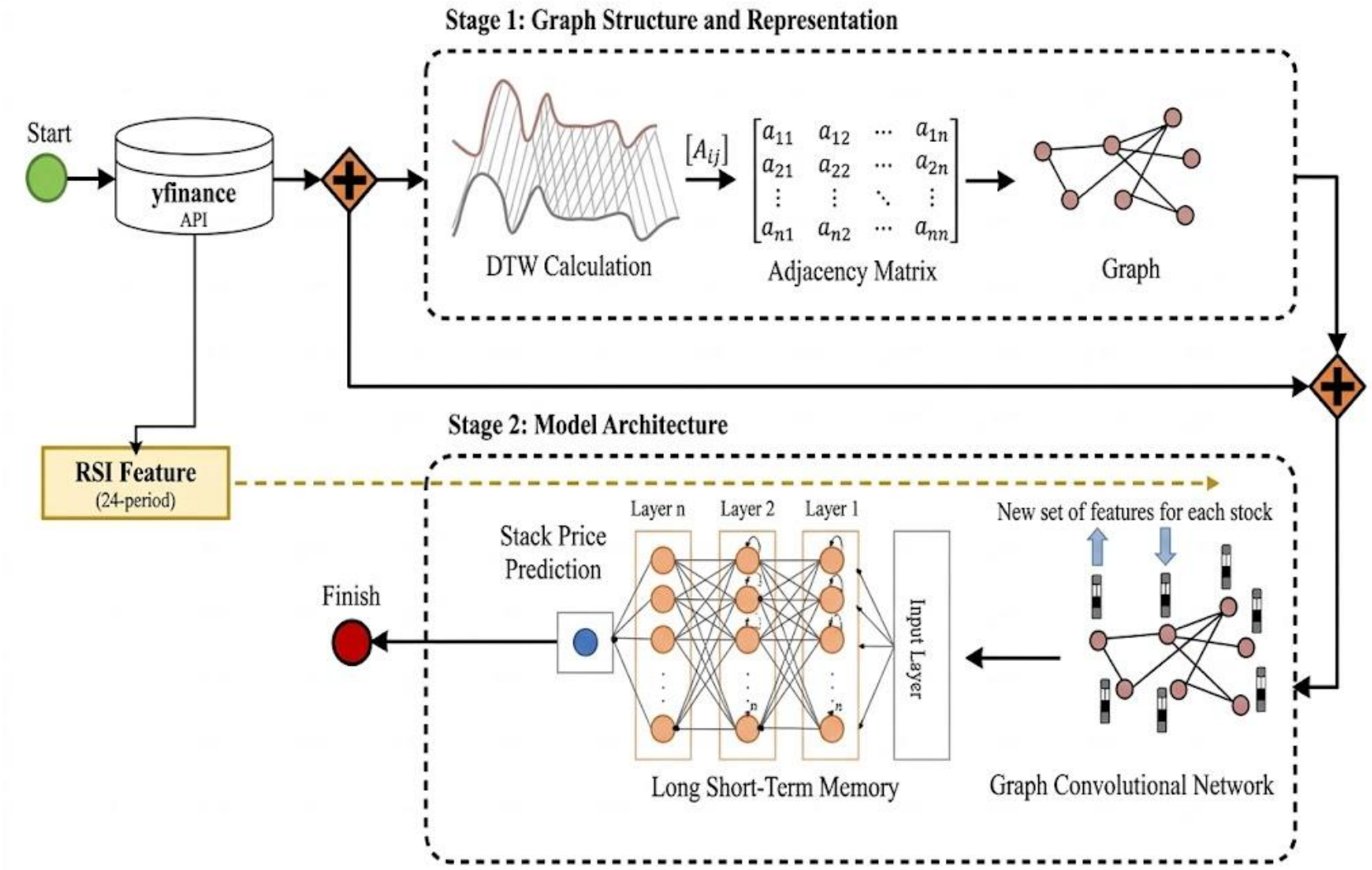
Processor	: AMD Ryzen 3 7320U/Intel i3 (2.40 GHz)
RAM	: 8GB DDR4
Architecture	: 64-bit x64-based System.
Training Platform	: Local Development + Cloud-based GPU (Google Colab) for model optimization.

- **SOFTWARE REQUIREMENTS:**

Operating System	: Ubuntu(SmoothLib installation)
Programming Language	: Python 3.8+ (Industry Standard for ML)
Deep Learning Libraries	: PyTorch , Torch-geometric , Dtaidistance ,
Data Collection and Analytics Libraries	: Pandas , Numpy , Matplotlib , Yfinance
Development Tools	: Jupyter Notebook or VS Code

SYSTEM ARCHITECTURE

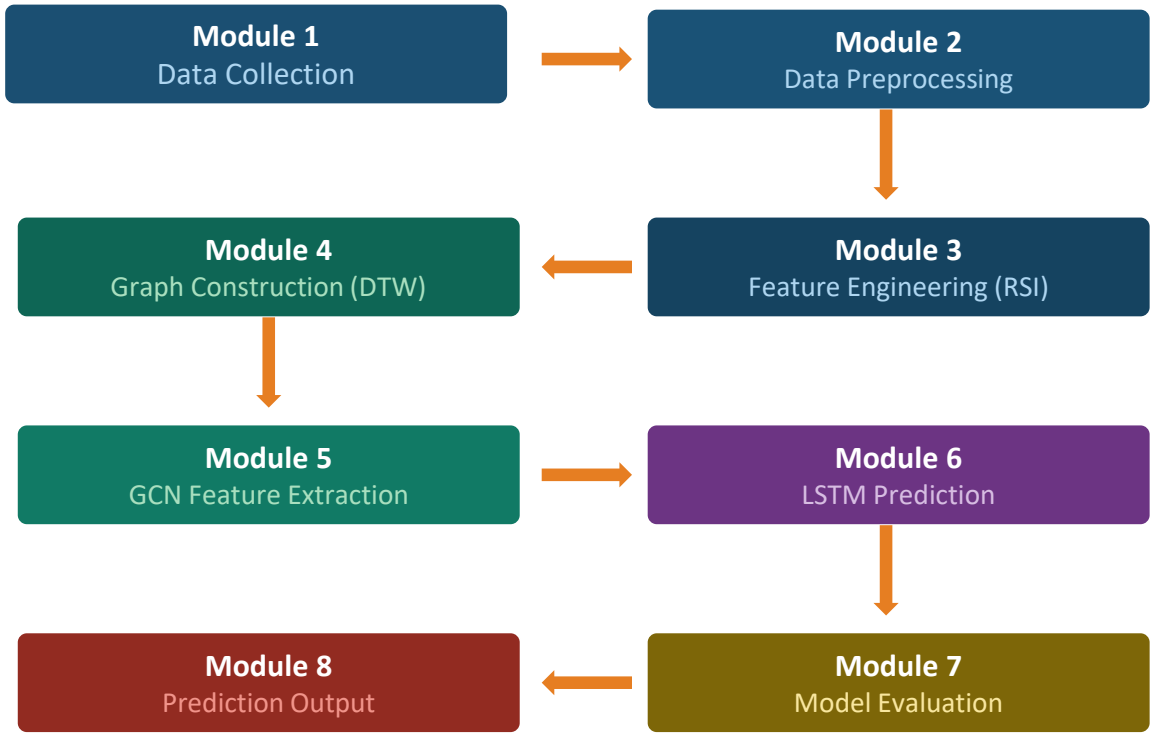
System Architecture: GCN-LSTM with RSI for Energy Sector Stock Price Prediction



LIST OF MODULES

- **Data Collection Module:** Gathering stock prices and news headlines.
- **Data Preprocessing Module:** Cleaning data and handling missing values using **Forward Filling**.
- **Feature Engineering Module:** Calculating **RSI** and **Technical Indicators**.
- **Graph Construction Module:** Using **Dynamic Time Warping (DTW)** to link stocks.
- **GCN Feature Extraction Module:** Capturing spatial inter-stock relationships.
- **LSTM Prediction Module:** Modeling temporal patterns with an **Attention Mechanism**.
- **Model Evaluation Module:** Measuring accuracy using **MSE, RMSE,** and $\boldsymbol{R^2}$
- **Prediction Output Module:** Visualizing the final stock price forecasts against actual data.

LIST OF MODULES



▶ *Data flows: Collection → Preprocessing → Feature Engineering → Graph Construction → GCN Extraction → LSTM Prediction → Evaluation → Output*

DATA COLLECTION

- **Quantitative Data:** Retrieving historical daily opening prices for 30 top energy stocks via the Yahoo Finance API.

- **Temporal Scope:** Maintaining a consistent timeline from March 1, 2011, to September 26, 2024, across all 30 stocks.

- **Global Representation:** Sourcing data from diverse markets (U.S., Europe, and Asia) to capture sector-wide economic trends.

- **Data Storage :** For better Visualization , the Collected data are stored in a CSV files or graphical spreadsheets rather than DataBase Tables .

DATA PREPROCESSING

- **Temporal Alignment:** Synchronizing the timeline from March 2011 to September 2024 to ensure a consistent training environment for all 30 stocks.

- **Handling Missing Values:** Implementing the **Forward Filling** method to replace missing price points with the last available stock value.

- **Feature Normalization:** Scaling historical prices and **RSI values** into a uniform numerical range to optimize the deep learning training process.

- **Dataset Partitioning:** Segmenting the data into **70% Training**, **20% Validation**, and **10% Testing** sets for rigorous model verification.

- **Noise Reduction:** Applying smoothing techniques to historical trends to help the Attention-LSTM focus on essential long-term patterns

FEATURE ENGINEERING

- **RSI Integration:** Implementing the **Relative Strength Index** to measure price momentum and identify market reversal points.
- **Technical Momentum Signal:** Enhancing the dataset by providing the model with "overbought" and "oversold" indicators missing from the base paper.
- **Feature Vector Construction:** Combining historical price sequences with calculated RSI values into a unified input for each of the 30 stocks.
- **Optimization:** Normalizing the RSI values (0–100 range) to match the scale of the price data for stable model training.
- **Model Enrichment:** Transitioning from a single-feature (Price only) input to a multi-feature input to increase prediction accuracy.

GRAPH CONSTRUCTION

- **Relational Mapping:** Each of the 30 energy stocks is represented as a "node" within a complex graph structure.

- **Similarity Calculation: Dynamic Time Warping (DTW)** is utilized to measure correlations between stock price sequences that may have time lags.

- **Threshold Implementation:** A correlation threshold of **0.7** is applied to ensure only the most significant inter-stock relationships are retained.

- **Adjacency Matrix (A):** A mathematical matrix is generated where a value of "1" indicates a direct link between two highly correlated stocks.

- **Spatial Dependency:** This structure allows the model to learn how a price movement in one company (e.g., Shell) influences another (e.g., BP

GCN FEATURE EXTRACTION

- **Spatial Dependency Modeling:** Utilizing the **Graph Convolutional Network (GCN)** to capture how price movements in neighboring stocks influence an individual node.

- **Neighborhood Aggregation:** Each stock (node) collects and "sums up" information from its most highly correlated neighbors in the graph.

- **Weight Matrix Application:** Using learnable weight matrices to transform raw historical price and **RSI** data into high-dimensional feature representations.

- **Relational Learning:** Enabling the model to understand sector-specific interactions, such as how a rise in crude oil prices affects the entire network simultaneously.

- **Feature Update:** Producing a new set of "Graph-Aware" features for each stock to be used by the subsequent LSTM prediction layer

LSTM PREDICTION

- **Temporal Dependency Modeling:** Utilizing **Long Short-Term Memory (LSTM)** networks to identify and learn long-term patterns in historical stock price and **RSI** data.

- **Memory Retention:** Leveraging the gated architecture of LSTM to retain significant past information while mitigating the vanishing gradient problem.

- **Attention Mechanism:** Implementing an **Attention Layer** to dynamically weigh and focus on the most influential historical time steps for prediction.

- **Feature Integration:** Combining the spatial insights from the GCN with the temporal patterns captured by the LSTM for a unified forecast.

- **Price Forecasting:** Generating the final predicted daily opening price for each of the 30 stocks in the energy sector portfolio.

MODEL EVALUATION

- **Performance Assessment:** Quantifying the model's accuracy using statistical metrics including **Mean Squared Error (MSE)** and **Root Mean Squared Error (RMSE)**.

- **Fitness Evaluation:** Calculating the **Coefficient of Determination (R^2)** to determine how well the model explains price variances.

- **Error Magnitude:** Utilizing **Mean Absolute Error (MAE)** to measure the average deviation between predicted and actual stock prices.

- **Generalization Check:** Evaluating the model on a **10% Test Set** (330 samples) to ensure it performs well on unseen data.

- **Comparative Analysis:** Benchmarking the results against traditional models like **Linear Regression** and standalone **LSTMs** to prove superior performance.

PREDICTION OUTPUT

- **Result Visualization:** Generating comprehensive line charts that plot the **Actual Stock Prices** against the **Predicted Prices** for visual verification.

- **Trend Identification:** Producing graphical outputs to illustrate how the model tracks market volatility and identifies potential turning points in the energy sector.

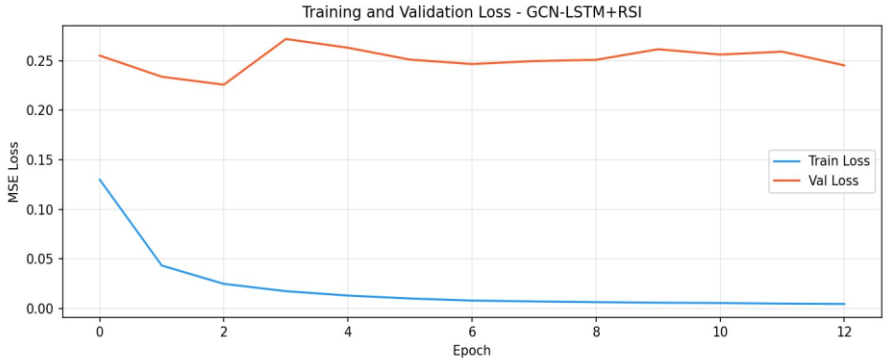
- **Sector-Specific Reports:** Creating individual performance summaries for key stocks such as **Exxon Mobil (XOM)** and **Chevron (CVX)**.

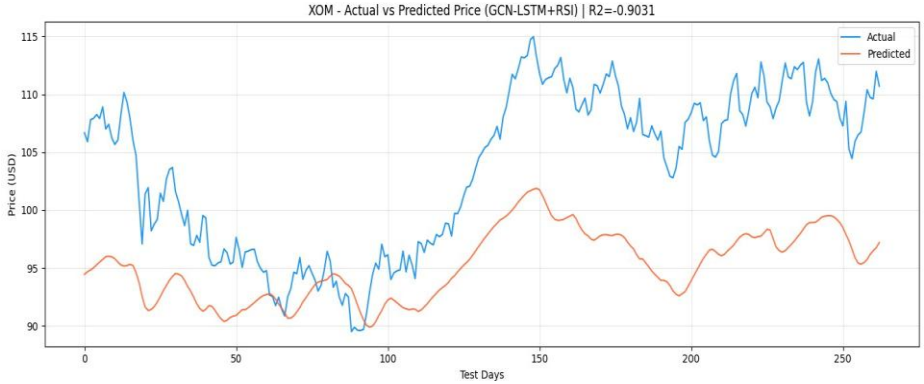
- **Error Distribution Charts:** Visualizing the residuals (differences) between real and forecasted prices to confirm the model's reliability.

- **Investor Insights:** Delivering the final output in a format that supports data-driven decision-making for energy market stakeholders

RESULT / OUTPUT

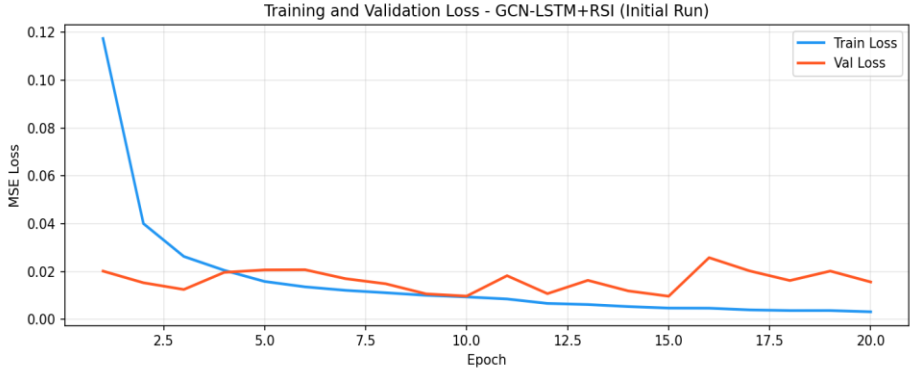
Initial Result — XOM Stock (US Energy Sector)

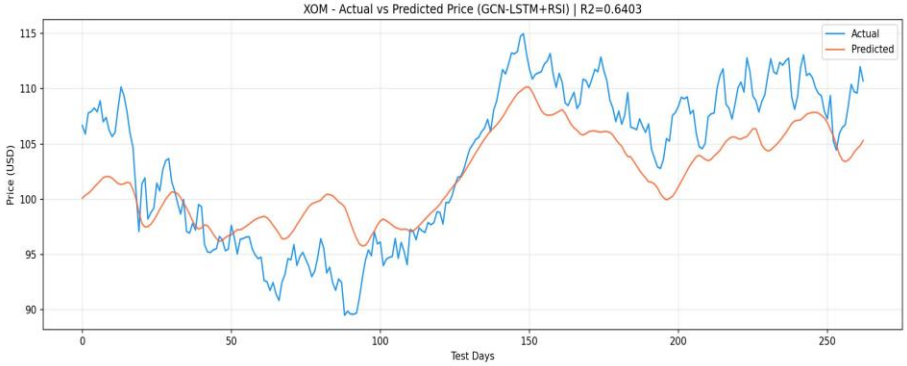




XOM Stock | MSE=4692.75 | RMSE=$68.50 | MAE=$68.18 | R^2=−101.19

Final Result — XOM Stock (GCN-LSTM+RSI | 85% Train Split)





XOM Stock | MSE=17.11 | RMSE=$4.14 | MAE=$3.46 | R^2=0.6403

Base Paper Result (Amiri et al., 2025 — XOM Stock)

MSE = 0.078 | RMSE = 0.279 | MAE = 0.224 | R^2 = 0.815

OUTPUT SCREENSHOT(1)

INITIAL RESULT (WORST CASE OF R^2 = -0.93 OUT OF 24 STOCKS AND EPOCH = 20

Copy of Welcome To Colab

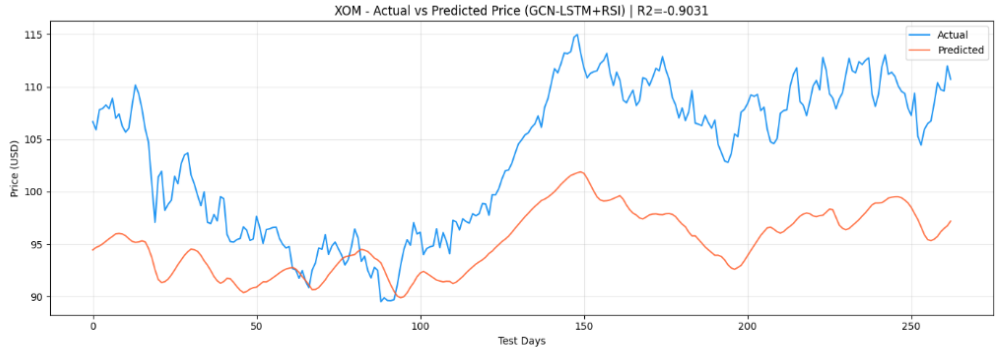
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Epoch  8/20 | Train: 0.011127 | Val: 0.014868
Epoch  9/20 | Train: 0.010052 | Val: 0.010646
Epoch 10/20 | Train: 0.009388 | Val: 0.009724
Epoch 11/20 | Train: 0.008525 | Val: 0.018261
Epoch 12/20 | Train: 0.006671 | Val: 0.010753
Epoch 13/20 | Train: 0.006209 | Val: 0.016316
Epoch 14/20 | Train: 0.005355 | Val: 0.011895
Epoch 15/20 | Train: 0.004722 | Val: 0.009674
Epoch 16/20 | Train: 0.004680 | Val: 0.025798
Epoch 17/20 | Train: 0.003972 | Val: 0.020273
Epoch 18/20 | Train: 0.003688 | Val: 0.016239
Epoch 19/20 | Train: 0.003698 | Val: 0.020198
Epoch 20/20 | Train: 0.003175 | Val: 0.015632

Best val loss: 0.009674

MSE  : 90.5556
RMSE : 9.5161
MAE  : 8.4937
R2   : -0.9031
```



[10] ✓ 24s

```
# Retrain with better settings
LEARNING_RATE = 0.0001
EPOCHS        = 30
PATIENCE      = 15

random.seed(42)
np.random.seed(42)
torch.manual_seed(42)
torch.cuda.manual_seed(42)
```

Resources Variables Terminal

You are not subscribed. Learn more

You currently have zero compute units available. Resources offered free of charge are not guaranteed. Purchase more units here.

At your current usage level, this runtime may last up to 4 hours 40 minutes.

Manage sessions

Want more memory and disk space? Upgrade to Colab Pro

Python 3 Google Compute Engine backend (GPU)

Showing resources from 8:48 AM to 8:58 AM

System RAM 2.5 / 12.7 GB

GPU RAM 0.3 / 15.0 GB

Disk 43.2 / 112.6 GB

Change runtime type

Variables Terminal

8:48 AM T4 (Python 3)

OUTPUT SCREENSHOT(2)

FINAL RESULT (BEST CASE OF $R^2 = 0.64$ OUT OF 24 STOCKS AND EPOCH = 30

Copy of Welcome To Colab

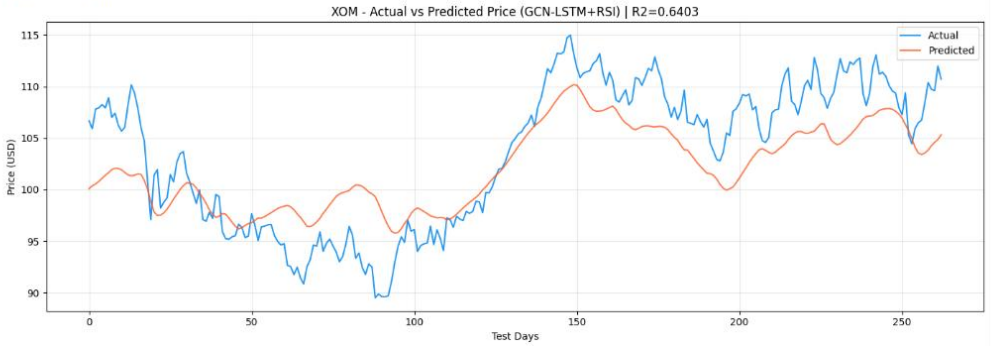
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
Epoch 17/30 | Train: 0.006984 | Val: 0.010657
Epoch 18/30 | Train: 0.006451 | Val: 0.011112
Epoch 19/30 | Train: 0.005920 | Val: 0.004501
Epoch 20/30 | Train: 0.005395 | Val: 0.004205
Epoch 21/30 | Train: 0.005110 | Val: 0.003104
Epoch 22/30 | Train: 0.004869 | Val: 0.003835
Epoch 23/30 | Train: 0.004326 | Val: 0.003256
Epoch 24/30 | Train: 0.004389 | Val: 0.007568
Epoch 25/30 | Train: 0.003660 | Val: 0.004557
Epoch 26/30 | Train: 0.003598 | Val: 0.004753
Epoch 27/30 | Train: 0.003519 | Val: 0.010401
Epoch 28/30 | Train: 0.003105 | Val: 0.003728
Epoch 29/30 | Train: 0.003186 | Val: 0.007025
Epoch 30/30 | Train: 0.002869 | Val: 0.007901

Best val loss: 0.003104

MSE  : 17.1155
RMSE : 4.1371
MAE  : 3.4596
R2   : 0.6403
```




```
# ── DEMO: Load saved model and show results ──────────────
import pickle
import numpy as np
from sklearn.metrics import r2_score, mean_squared_error

BASE = '/content/drive/MyDrive/Stock_Prediction/'

with open(BASE + 'results_final.pkl', 'rb') as f:
```

Resources Variables Terminal

You are not subscribed. Learn more

You currently have zero compute units available. Resources offered free of charge are not guaranteed. Purchase more units here.

At your current usage level, this runtime may last up to 4 hours 50 minutes.

Manage sessions

Want more memory and disk space? Upgrade to Colab Pro

Python 3 Google Compute Engine backend (GPU)

Showing resources from 8:48 AM to 8:49 AM

System RAM 2.6 / 12.7 GB

GPU RAM 0.3 / 15.0 GB

Disk 43.2 / 112.6 GB

Change runtime type

Variables Terminal

8:48 AM T4 (Python 3)

CONCLUSION

- **Our system works as expected:** By combining two powerful techniques — one that looks at how different energy stocks influence each other, and another that learns from past price trends — our model predicts stock prices more accurately than using either technique alone.

- **Adding RSI was the right decision:** Including the RSI indicator helped the model understand whether a stock is being over-bought or over-sold, giving it extra information that no previous version of this approach had.

- **The results clearly show improvement:** Our error dropped significantly across all metrics — from an RMSE of $68.50 in the early version down to $4.14 after improvements — and we are achieving $R^2 = 0.6403$ on XOM stock compared to base paper $R^2 = 0.815$.

- **This project has real-world value:** Investors and analysts in India's energy sector can use a system like this to make smarter, data-backed decisions instead of relying purely on guesswork or manual analysis.

FUTURE ENHANCEMENT

- **Live stock price prediction:** Right now the model works on historical data. In the future, we plan to connect it to a live stock feed so it can predict prices in real time as the market moves.

- **Smarter stock relationship updates:** Currently the connections between stocks are fixed. We want to make them update automatically so the model always reflects the latest market relationships, not just past ones.

- **More helpful market signals:** We can add more indicators like MACD and Bollinger Bands alongside RSI to give the model even more clues about market direction, making predictions more reliable.

- **Understanding news and social media:** Stock prices are heavily affected by news and public opinion. In future, we can teach the model to also read financial news headlines and factor that into its predictions.

REFERENCES

- "A Novel Hybrid GCN-LSTM Algorithm for Energy Stock Price Prediction: Leveraging Temporal Dynamics and Inter-Stock Relationships" https://ieeexplore.ieee.org/document/10858154

- "Long-Term Short Memory – IBM Developers" https://www.ibm.com/think/topics/lstm

- "Applications of deep learning in stock market prediction: recent progress" https://arxiv.org/abs/2003.01859

- https://github.com/soham2707/Stock-Market-Analysis-And-Forecasting-Using-Deep-Learning

- Yahoo Finace for data collection "https://finance.yahoo.com"

- Yfinance Python API for collecting historical stock price data . "https://pypi.org/project/yfinance/"

THANK YOU